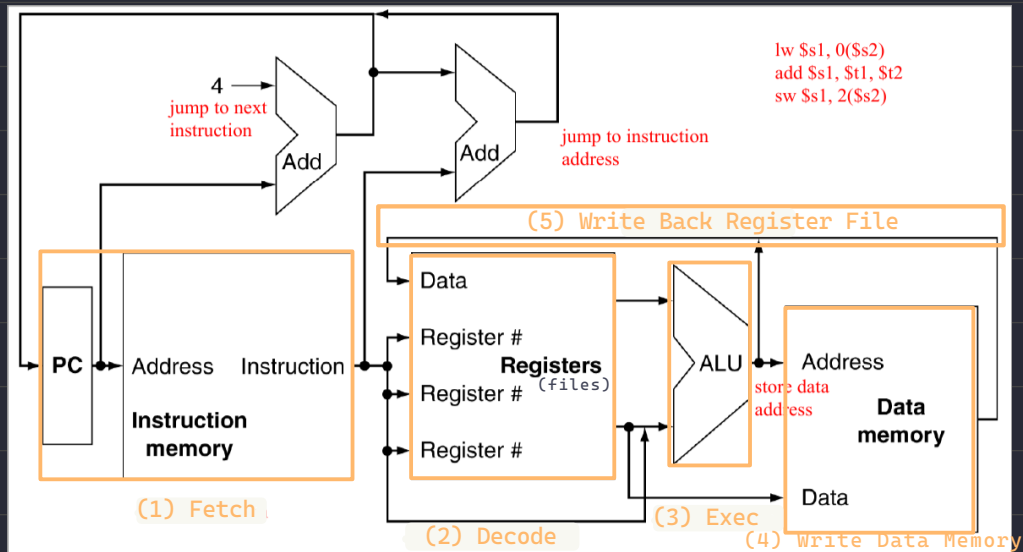


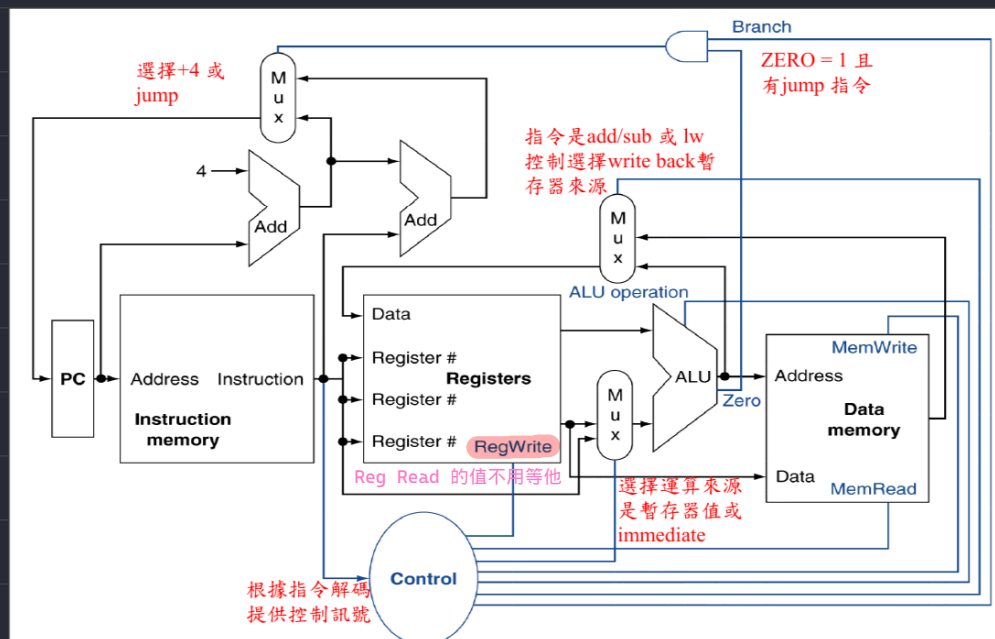
## 4.1 Introduction

### Abstract view of CPU



### Control Line

必考！畫出現路上的 data path



### The Critical Path

找最長的路徑 ;D , 注意到 Register File 的 Write Address 可以不用 READY, Data out 就可以用了

clock skew: 時鐘訊號到達數位電路各元件耗用時間差

### Exercise (Q4.3) Cost Performance Problem

$$\text{Speedup} = \frac{\text{Instructions}_{old}}{\text{Instructions}_{new}} \times \frac{\text{Cycle time}_{old}}{\text{Cycle time}_{new}}$$

$$\text{Cost/Performance} = \frac{\text{Cost}_{old}}{\text{Cost}_{new}} \div \frac{\text{Speed}_{old}}{\text{Speed}_{new}}$$

$$\begin{aligned} \text{Speedup} &= (\text{new speed})/(\text{old speed}) \\ &= (1/\text{new time})/(1/\text{old time}) \\ &= (\text{old time})/(\text{new time}) \end{aligned}$$

4.2 Logic Design Conventions

JK Flipflop

| Inputs |   |     | Outputs |    |           |
|--------|---|-----|---------|----|-----------|
| J      | K | CLK | Q       | Q' | Remark    |
| 0      | 0 | ↑   | NC      | NC | No Change |
| 0      | 1 | ↑   | 0       | 1  | RESET     |
| 1      | 0 | ↑   | 1       | 0  | SET       |
| 1      | 1 | ↑   | Q'      | Q  | Toggle    |

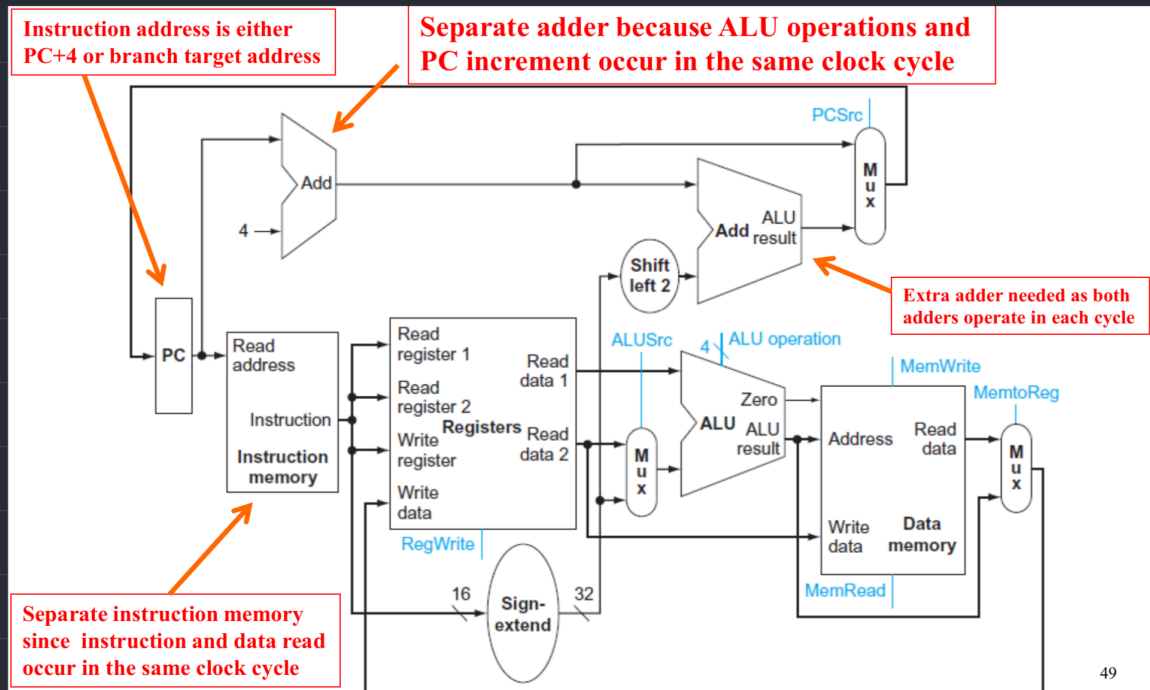
D Latch

| Input |   | Output |    | Remark      |
|-------|---|--------|----|-------------|
| C     | D | Q      | Q' |             |
| 0     | × | NC     | NC | Not change  |
| 1     | 0 | 0      | 1  | Latch RESET |
| 1     | 1 | 1      | 0  | Latch SET   |

D Flip-flop

| Input |   | Output |    | Remark      |
|-------|---|--------|----|-------------|
| Clk   | D | Q      | Q' |             |
| 0 1↓  | × | NC     | NC | Not change  |
| ↑     | 0 | 0      | 1  | Latch RESET |
| ↑     | 1 | 1      | 0  | Latch SET   |

### 4.3 Building a Datapath



### Exercise (Q4.4)

MIPS 不真正需要 unconditional branches → beq zero, zero, Label 可執行此功能

1. If the only thing we need to do in a processor is fetch consecutive (連續的) instructions(Figure 4.6), what would the cycle time be?

ANS: I-Mem

2. Consider a datapath similar to Figure 4.11, but only has one type of instruction: unconditional PC-relative branch (類似jump, 但會+PC, 事實上MIPS不需要這種指令). What would the cycle time

3. Repeat 4.4.2, but this time we need to support only conditional PC-relative branches.

## Datapath with Control II



## 4.5 An Overview of Pipelining

### Hazards

Structural hazards: 前後指令讀寫同一個記憶體

Data hazards: 後指令用前結果

Control hazards: 判斷結果才知道要執行哪一條指令

## Data Hazards

先讀後寫(WAR) → 沒事

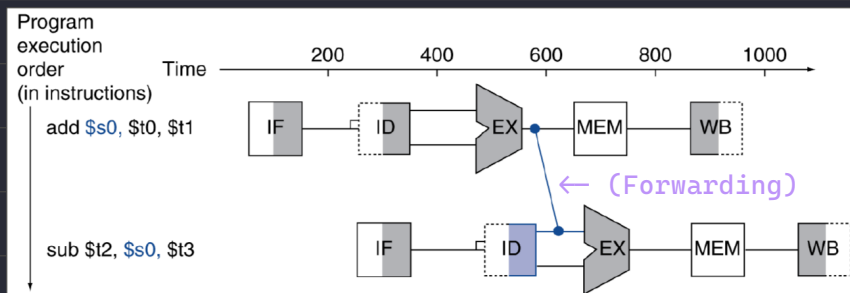
```
add $t3, $t2, $s1 ; $t3 ← $t2 + $s1
add $s1, $t4, $t5 ; $s1 ← $t4 + $t5
```

寫後寫(WAW) → 沒事

```
lw $s1, 0($t2) ; $s1 ← MEM($t2)
add $s1, $t1, $t4 ; $s1 ← $t1 + $t4
```

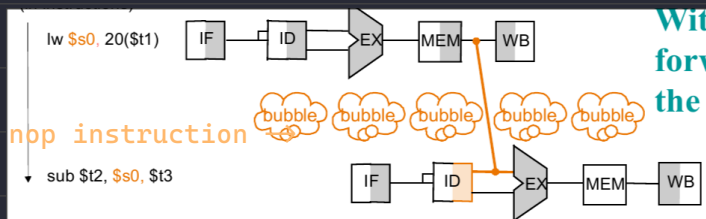
先寫後讀(RAW) → 出事

Case 1: forwarding or bypassing: 設計線路直接將 ALU 結果拿過去



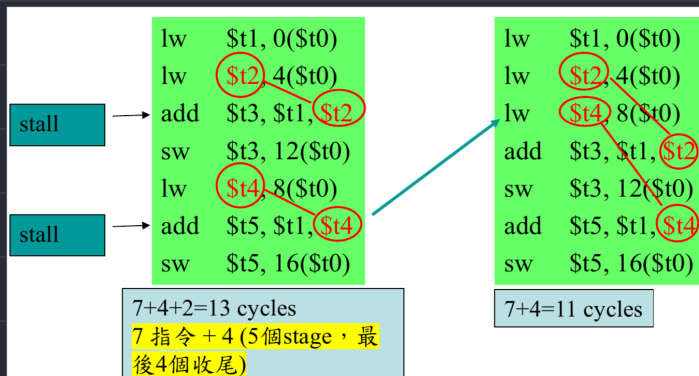
```
add $s0, $t0, $t1
sub $t2, $s0, $t3
```

Case 2: forwarding or bypassing: 加入等待時間 (bubble) 等結果

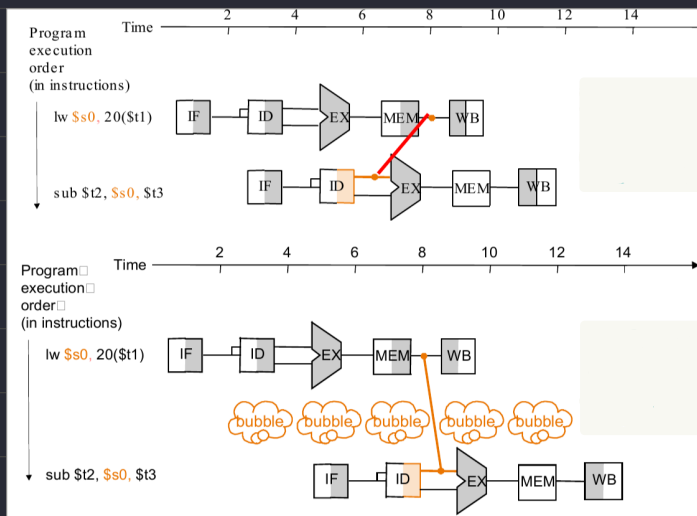


```
lw $s0, 20($t1) ; $s0 ← MEM(20+$t1)
sub $t2, $s0, $t3 ; $t2 ← $s0 - $t3
```

Case 3: Reorder code to avoid use of load result in the next instruction



## Case 4: Load-Use Data Hazard

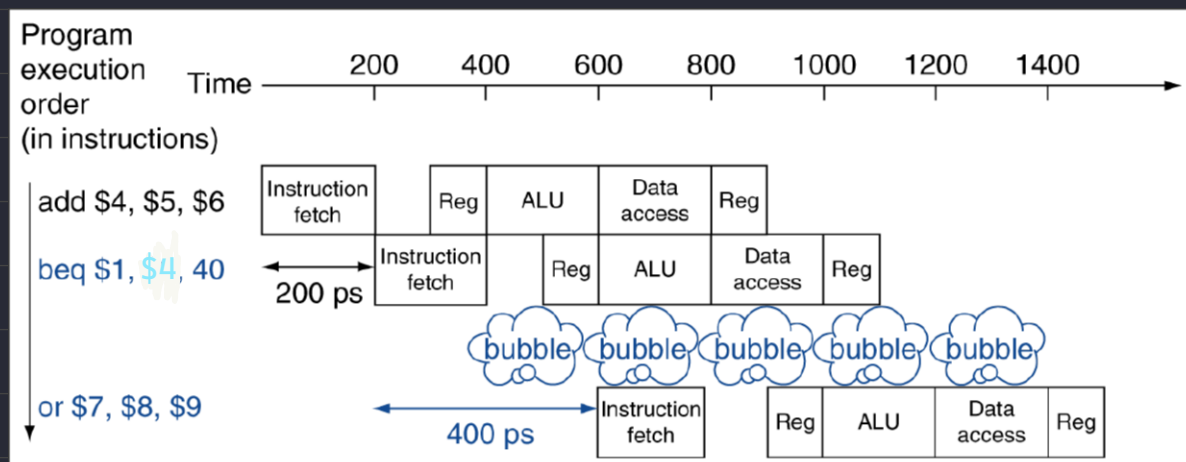


現在的時間無法讀取到未來的值

配合 forwarding 電路把資料拉回來

## Control Hazards

### Bubble Solution



### Prediction Solution

#### Static branch prediction (deprecated)

預設結果是 Not taken (不跳轉) / Taken (跳轉)

→ predict correct: Fetch instruction after branch, with no delay

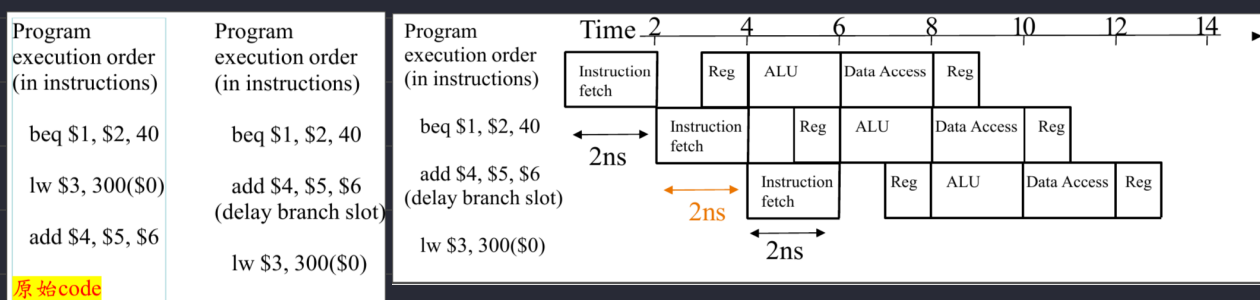
→ predict wrong: Only stall

#### Dynamic branch prediction: hardware-based

儲存歷史的跳轉紀錄，最後根據經驗判斷是否跳轉

#### Delayed branch (deprecated)

編譯器賄選一條不受 branch 影響的指令放到 branch 後面，把原本bubble 的時間拿來做其他事情



### Types of forwarding

Without Forwarding : 什麼多的線都沒有

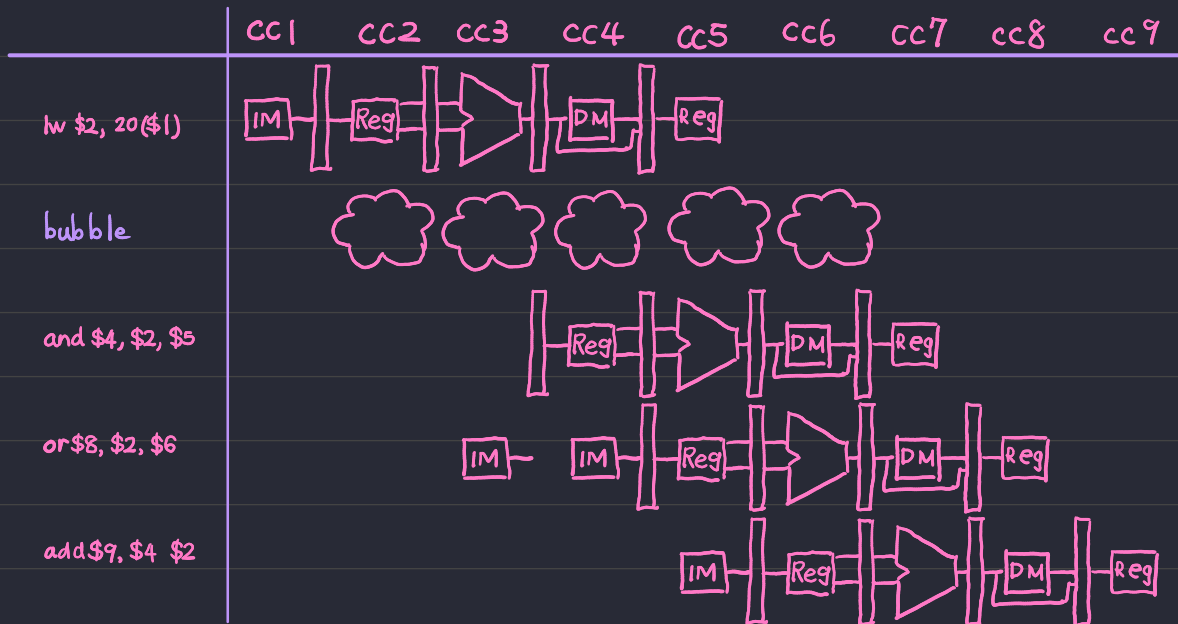
With Full Forwarding : ALU-ALU + MEM-EXEC 的 forward

With ALU-ALU Forwarding Only : 如其名

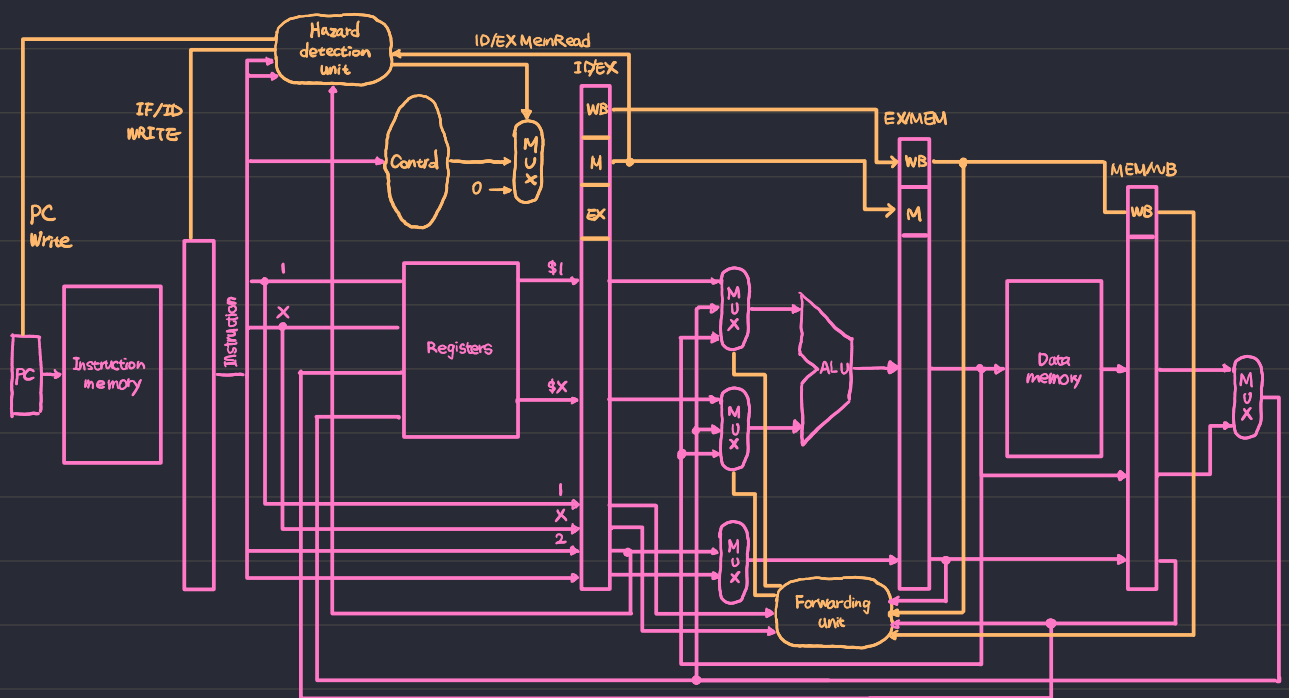




## Zuvio week1501



## Zuvio week1502





考試會考：

# Pipeline with Exceptions

